

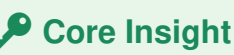
TreeDiff: AST-Guided Code Generation with Diffusion-based LLMs (DLLMs)

🔗 TreeDiff = first AST-aware masking for diffusion code generation → +13.3% on HumanEval+, narrows gap with autoregressive models.



Diffusion-based LLMs (DLLMs) struggle with **syntactic precision** in code generation. Standard random token masking:

- Ignores syntactic boundaries during denoising
- Fails to capture long-range hierarchical dependencies
- Treats all tokens equally — but they're not!

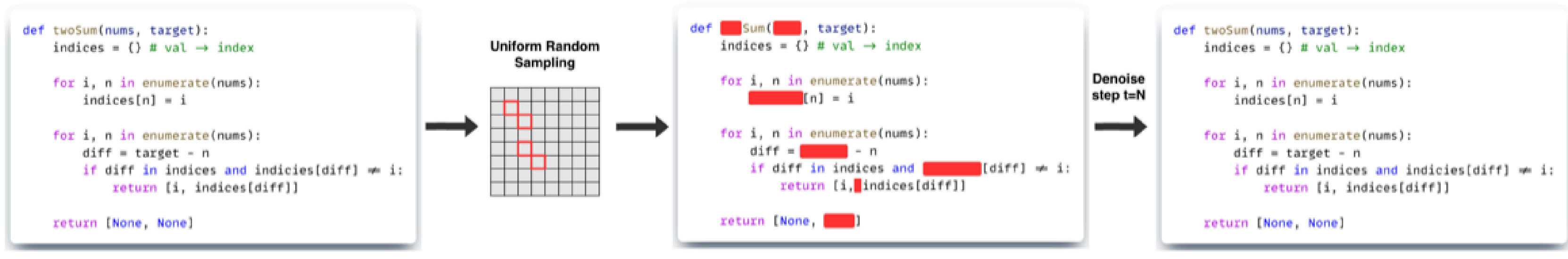


"For code, noise should not be purely stochastic — structural priors enable DLLMs to learn program-level dependencies."

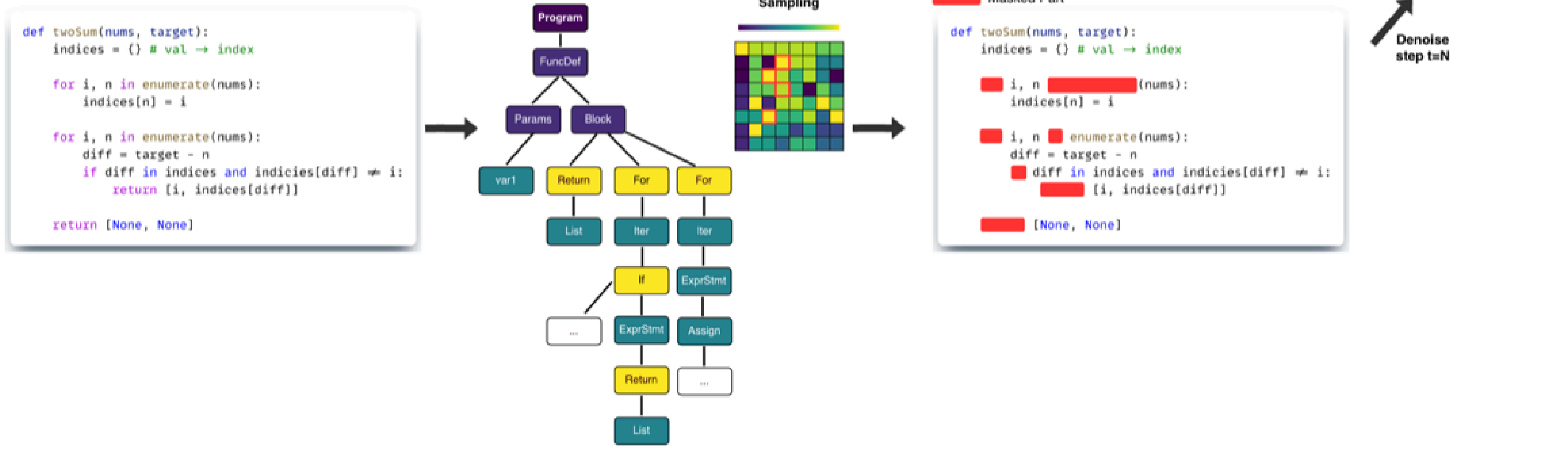


TreeDiff parses code into an **Abstract Syntax Tree (AST)** and uses it to guide which tokens to mask — targeting structurally critical nodes.

(A) Random tokenized masking for training



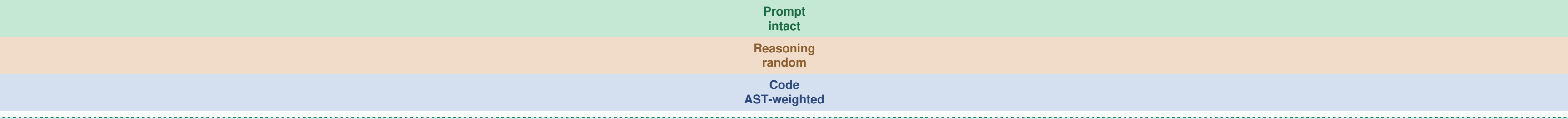
(B) Our approach: AST-masked method for training



TreeDiff augments masked diffusion with **AST-aware policies**: the syntax tree guides which tokens to mask and how to weight the denoising loss.



Each diff region gets a tailored masking policy:



Loss weights scale with AST node depth: **deeper nodes receive higher weight** than surface tokens.

$w(\text{node}) = 1 + \alpha \cdot \text{depth}(\text{node})$				
Category	Examples	Weight		Purpose
Structure	Imports, Constants	0.15		Preserve skeleton
Data Flow	Assigns, Calls	0.42		Moderate priority
Conditionals	if, elif, else	0.58		Core logic
Control Flow	for, while, return	0.60		⚡ Highest priority

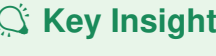


+13.3%									
42.1									
Model		T = 256				T = 512			
		HE	HE+	MBPP	MBPP+	HE	HE+	MBPP	MBPP+
Autoregressive Models†									
DeepSeek-Coder-33B		50.6	44.5	80.4	70.1	50.6	44.5	80.4	70.1
CodeQwen1.5-7B		51.8	45.7	73.5	60.8	51.8	45.7	73.5	60.8
CodeLlama-13B		42.7	38.4	63.5	52.6	42.7	38.4	63.5	52.6
Diffusion-based LLMs (LLaDA Backbone)									
LLaDA-Instruct		39.6	34.8	51.9	43.1	36.6	31.7	39.4	31.7
+ Random Masking		39.6	35.4	52.9	43.9	38.4	32.3	41.5	32.8
Ours									
+ TreeDiff		42.1	37.2	52.9	44.2	40.2	36.6	42.3	33.3

† From EvalPlus Leaderboard. HE = HumanEval, HE+ = HumanEval+.



HumanEval		T=256		T=512	
Strategy					
Random Masking		39.6 / 35.4		38.4 / 32.3	
AST (Span)		40.9 / 36.6		40.2 / 36.6	
AST (Node)		42.1 / 37.2		40.3 / 36.6	
MBPP		T=256		T=512	
Strategy					
Random Masking		52.9 / 43.9		41.5 / 32.8	
AST (Span)		51.6 / 42.9		39.7 / 31.5	
AST (Node)		52.9 / 44.9		42.3 / 33.3	



Key Insight
Random masking outperforms span masking on code diffs. Spans often split syntactic units (e.g., an if-condition), while random masking preserves more complete AST nodes. TreeDiff's *token-level* AST masking surgically targets high-influence nodes while preserving context.

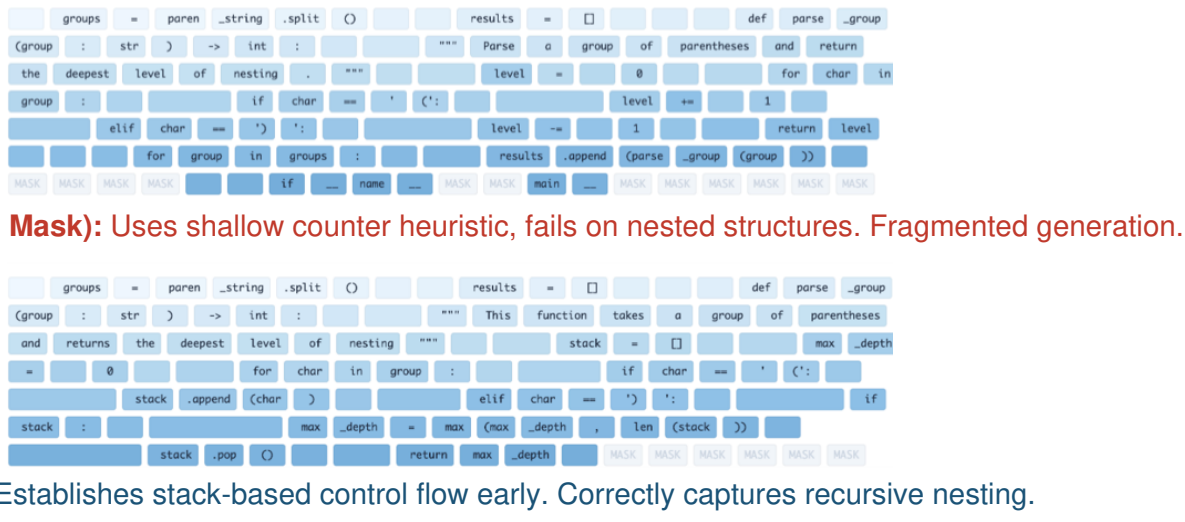


Method	T=312	T=1024
LLaDA + span mask	35.6	34.2
TreeDiff	42.1	41.8

TreeDiff maintains stable performance (~0.1%) as T increases 256→512→1024, while baselines degrade by ~2.1% due to fragmented syntax trees.



Task: Max nesting depth of parentheses (HumanEval/6), Step 110/256.



HumanEval/6 — Nested parentheses depth:

Vanilla MD (Random Mask)
count = 0 for char in group: if char == '(': count += 1 # fails to track # nested peak depth
TreeDiff (AST-guided)
for char in group: if char == '(': stack.append(char) max_depth = max max_depth, len(stack) elif char == ')': if stack: stack.pop()
HumanEval/6: Maximum nesting depth of parentheses



- **First AST-aware masking** for DLLMs in code generation
- Novel AST-aware masking and loss weighting that amplifies structural signal
- Achieve **+13.3%** relative HumanEval+ and **42.1** Pass@1 with only 50 denoising steps — lower FLOPs than 256-step autoregressive decoding
- Practical recipe for adapting pretrained masked diffusion LMs to structured seq2seq tasks without catastrophic forgetting

TreeDiff addresses a fundamental limitation: random masking ignores code structure. By aligning corruption with AST hierarchy, the model learns programming language compositionality, reconstructing programs that respect grammatical boundaries.